Лабораторная работа 2 Контракт данных и предотвращение утечки

Версия: 2026-2027 учебный год

Формат: командная работа по 2-5 человек с индивидуальной сдачей

Предшествующая работа: ЛР1 Репозиторий команды и воспроизводимая среда

Продолжение: результаты ЛР2 используются в ЛР3 при построении версионируемого DVC-конвейера

1 Назначение работы

Команда должна превратить учебный набор данных из безымянного CSV-файла в управляемый артефакт. Для этого требуется зафиксировать происхождение и допустимое использование данных, создать понятную человеку карточку набора, описать машинно-проверяемый контракт и реализовать автоматическую проверку качества. Валидатор должен не только сообщать об ошибке, но и формировать воспроизводимый отчет с версией контракта и контрольной суммой входного файла.

В ЛР2 не требуется настраивать DVC, удаленное хранилище или полный обучающий pipeline. Эти задачи относятся к ЛР3. Здесь оценивается готовность данных к безопасному включению в такой pipeline.

2 Результаты обучения

После выполнения работы каждый студент должен уметь:

- объяснить различие между карточкой набора данных и машинным контрактом;
- проследить происхождение набора от источника до файла в проекте;
- определить обязательные поля, типы, допустимые значения, диапазоны и правила пропусков;
- реализовать автоматическую проверку схемы, полноты, уникальности и диапазонов;
- получить SHA-256 проверенного файла и связать его с отчетом валидации;
- зафиксировать момент прогноза и разделить train, validation и test строго по времени;
- объяснить, какие данные запрещено публиковать в Git;
- воспроизвести успешную и отрицательную проверку независимо от других участников команды.

3 Командный результат

К завершению ЛР2 команда предоставляет:

1. заполненную карточку набора `docs/data-card.md`;
2. версионируемый контракт `data/contract.json`;
3. валидатор `src/mlops_flight_delay/validate_data.py` с ненулевым кодом завершения при нарушении;
4. тесты контракта, включая не менее двух намеренно испорченных вариантов данных;
5. JSON-отчет успешной проверки с версией контракта, SHA-256, числом строк и результатами отдельных правил;
6. документированные границы хронологических train, validation и test и меры против утечки;
7. безопасный разрешенный образец данных вместо полного или закрытого набора;
8. issue и pull request с review участника, который не является автором изменения;
9. заполненную индивидуальную карточку `docs/evidence/lr2-evidence-card.md` для каждого участника;
10. принятую контрольную точку `lr2-v1` в виде Git tag или GitHub Release после успешного CI.

4 Исходные данные и границы работы

В стартовом проекте используется файл `data/flights_sample.csv` из 24 строк. Он нужен для smoke-проверок; основная работа выполняется на подготовленном преподавателем фрагменте BTS объёмом 20–50 тысяч строк или на согласованном эквиваленте:

Поле	Роль	Смысл	Учебное ограничение
`FlightDate`	время события	дата рейса и ключ хронологического разбиения	формат `YYYY-MM-DD`, строки отсортированы по возрастанию
`DepHour`	признак	час отправления по местному времени	целое число от 0 до 23
`IsWeekend`	признак	признак выходного дня	только 0 или 1
`ArrDelay`	источник цели	задержка прибытия в минутах	целое число от -180 до 1440

Целевой класс формируется условием `ArrDelay >= 15`. Момент прогноза — за 60 минут до планового отправления. Сам `ArrDelay` и любые факты, появившиеся позже момента прогноза, нельзя подавать модели как признаки: это прямая утечка. Учебный файл синтетический, не содержит персональных данных и нужен для проверки инфраструктуры, а не для выводов о реальных рейсах.

Если преподаватель выдал другой набор, команда обязана адаптировать контракт и карточку под него. Нельзя механически сохранить диапазоны из примера. Для реальных данных укажите владельца, способ получения, дату или период, лицензию, ограничения использования, чувствительные поля и порядок удаления.

5 Роли команды в ЛР2

Роль	Ответственность	Индивидуальное подтверждение
Data steward	уточняет происхождение, лицензию, владельца и допустимое использование	разделы карточки данных и ссылка на источник или решение команды
Contract engineer	проектирует поля, типы, обязательность, диапазоны и версию контракта	diff `data/contract.json` и объяснение двух правил
Quality engineer	реализует валидатор и отрицательные тесты	код проверки, тест или диагностический отчет
Leakage reviewer	проверяет признаки, цель и способ разделения	запись решения, review или тест против утечки
Repository and documentation reviewer	контролирует Git-исключения, README, evidence cards и итоговый PR	содержательное review и проверяемое изменение документации

В команде из 2-4 человек роли совмещаются. В команде из пяти участники получают отдельные зоны ответственности. Название роли само по себе не подтверждает вклад: необходим собственный commit, review, тест, отчет или документируемое решение.

6 Подготовка репозитория

Начинайте от принятого результата ЛР1. Обновите локальную ветку и создайте отдельную ветку задачи:

```
git switch main
git pull --ff-only
git switch -c lab2/data-contract
python -m pip install -e ".[dev]"
```

Проверьте, что базовые тесты проходят до изменения данных:

```
python -m ruff check .
python -m pytest -q
```

Создайте issue с критериями готовности ЛР2. Назначьте владельцев артефактов и reviewer. Не добавляйте в issue секретные ссылки, токены доступа или фрагменты закрытых данных.

7 Шаг 1 Инвентаризация и происхождение данных

До написания кода ответьте на вопросы:

1. Кто создал или предоставил данные?
2. Когда и каким способом они получены?
3. На каком основании команда может их использовать и публиковать?
4. Содержат ли они персональные, коммерчески чувствительные или иные ограниченные сведения?
5. Какой файл разрешено хранить в Git?
6. Кто отвечает за обновление и удаление набора?
7. Как проверить, что используемый файл не изменился незаметно?

Для полного реального набора зафиксируйте внешнее расположение или идентификатор версии, но не помещайте закрытые данные в репозиторий. В Git можно оставить небольшой обезличенный образец, если это разрешено владельцем и лицензией. Добавьте каталоги `data/raw/` и `data/processed/` в `.gitignore`.

Получите контрольную сумму учебного файла:

```
python -c "from pathlib import Path; import hashlib; p=Path('data/flights_sample.csv'); print(hashlib.sha256(p.read_bytes()).hexdigest())"
```

Контрольная сумма не доказывает качество или законность данных. Она позволяет однозначно связать отчет с конкретными байтами файла.

8 Шаг 2 Карточка набора данных

Карточка `docs/data-card.md` объясняет набор человеку. Она должна содержать:

- название, назначение и запрещенные сценарии использования;
- происхождение, владельца, способ получения и период;
- лицензию или иное основание использования;
- состав полей и способ построения целевой переменной;
- размер, единицы измерения и известные пропуски;
- чувствительные данные и примененное обезличивание;
- ограничения репрезентативности и известные смещения;
- стратегию разделения и риск утечки;
- правила обновления, версионирования и удаления.

Карточка не должна повторять весь JSON-контракт. Ее задача - объяснить контекст и риски. Контракт, напротив, служит точным исполнимым источником требований.

9 Шаг 3 Машинный контракт данных

Контракт хранится в `data/contract.json`. Минимальная структура:

```
{
"contract_version": "1.0.0",
"dataset": "flight-delay-training-sample",
"minimum_rows": 20,
"allow_extra_columns": false,
"duplicate_rows": "forbid",
"columns": {
"FlightDate": {
"role": "event_time",
"type": "date",
"format": "%Y-%m-%d",
"nullable": false
},
```

```
"DepHour": {
"role": "feature",
"type": "integer",
"nullable": false,
"minimum": 0,
"maximum": 23
},
"IsWeekend": {
"role": "feature",
"type": "integer",
"nullable": false,
"allowed_values": [0, 1]
}
}
}
```

Каждое правило должно иметь предметный смысл. Например, диапазон часа от 0 до 23 следует из единицы измерения, а не из минимума и максимума текущего файла. Если правило основано только на наблюдаемой статистике, обозначьте его как предупреждение или обоснуйте запас.

Версию контракта меняйте осознанно:

- исправление описания без изменения проверки - patch;
- совместимое добавление необязательного поля - minor;
- удаление или переименование поля, изменение типа или смысла - major.

Это учебное правило версионирования должно быть зафиксировано командой в README или карточке данных.

10 Шаг 4 Автоматическая валидация

Валидатор стартового проекта запускается так:

```
python -m mlops_flight_delay.validate_data \
--data data/flights_sample.csv \
--contract data/contract.json \
--report reports/validation/lr2.json
```

В PowerShell можно записать команду в одну строку. Успешная проверка должна завершаться кодом 0, неуспешная - ненулевым кодом. Отчет должен содержать не менее следующих полей:

Поле отчета	Назначение
`contract_version`	связывает результат с правилами проверки
`dataset`	идентифицирует логический набор
`data_file`	показывает проверенный путь
`sha256`	идентифицирует содержимое файла
`rows` и `columns`	фиксируют базовую форму набора
`passed`	дает итоговый результат для CI
`checks`	объясняет прохождение или отказ по каждому правилу

Минимальный набор автоматических проверок:

- наличие обязательных и отсутствие неожиданных полей;
- минимальное число строк;
- запрет пропусков в обязательных полях;
- преобразуемость в заявленный тип и целочисленность;
- допустимые диапазоны и категории;

- запрет полных дубликатов;
- ненулевой код завершения при любом нарушении.

Не ловите все исключения с последующим успешным завершением. Ошибка чтения файла, поврежденный контракт или нарушение данных должны быть видны пользователю и CI.

11 Шаг 5 Негативные сценарии

Положительный тест доказывает только то, что текущий пример проходит. Для приемки создайте во временном каталоге не менее двух испорченных вариантов:

1. значение `DepHour=25`, нарушающее верхнюю границу;
2. отсутствие `ArrDelay` или недопустимое значение `IsWeekend=2`.

Не изменяйте эталонный `flights_sample.csv` ради демонстрации. Тест должен построить временный файл, вызвать валидатор и проверить, что нужное правило стало неуспешным. Дополнительно рекомендуются сценарии с пропуском, дробным значением целочисленного поля, неизвестным столбцом, полным дубликатом и недостаточным числом строк.

Запустите проверки:

```
python -m pytest -q tests/test_data_contract.py
python -m ruff check src tests
```

В CI валидатор должен выполняться до обучения модели. Если данные не соответствуют контракту, дальнейшие этапы не запускаются.

12 Момент прогноза, разбиение и предотвращение утечки

Сначала зафиксируйте prediction timestamp: прогноз строится за 60 минут до планового отправления. Для каждого потенциального признака укажите event timestamp и available timestamp. Признак допускается только тогда, когда его значение гарантированно известно к prediction timestamp.

После сортировки по времени задайте три непересекающихся периода:

- train — самый ранний период, на нём обучаются преобразования и модель;
- validation — следующий период, на нём выбираются решение и пороги;
- test — самый поздний период, он используется один раз для итоговой независимой оценки.

В стартовом образце используется пропорция 60/20/20. Для данных BTS команда фиксирует календарные границы и количество строк в каждой части. Перемешивание и случайная стратификация запрещены. Статистики нормализации, словари категорий и заполнение пропусков обучаются только на train. `ArrDelay` используется только для построения отложенной цели и не входит в `FEATURES`.

В validation report добавьте границы периодов, число строк, долю положительного класса и проверку `max(train_time) < min(validation_time) <= max(validation_time) < min(test_time)`.

13 Git и безопасность данных

Запрещено добавлять в репозиторий:

- полные закрытые или лицензионно ограниченные наборы;
- персональные данные и их необезличенные фрагменты;
- секреты, токены, строки подключения и подписанные URL;
- выгрузки с неизвестным происхождением;
- временные обработанные копии, если они воспроизводимы;
- файлы, превышающие правила репозитория и не предназначенные для Git.

Если чувствительный файл уже попал в commit, одного удаления в новом commit недостаточно: значение остается в истории. Остановите публикацию, отзовите скомпрометированные секреты, сообщите

преподавателю или владельцу и примените согласованную процедуру очистки истории.

14 Pull request и контрольная точка

Pull request ЛР2 должен включать контракт, карточку данных, валидатор, тесты, README и индивидуальные evidence cards. В описании укажите:

- источник и разрешенное использование набора;
- версию контракта;
- команду успешной проверки;
- два отрицательных сценария;
- SHA-256 принятого образца;
- выбранную стратегию разделения;
- известные ограничения и отложенные задачи для ЛР3.

Reviewer сверяет правила с предметным смыслом, запускает проверки локально и оставляет содержательный комментарий. После успешного CI и merge создайте tag `lr2-v1`. Отчет валидации должен быть воспроизводим по tag, даже если команда не хранит генерируемый JSON в Git.

15 Правила индивидуальной сдачи

Командный pull request необходим, но не заменяет индивидуальную защиту. Каждый участник копирует шаблон `docs/evidence/lr2-evidence-card.md`, называет файл по правилам преподавателя и заполняет его самостоятельно.

15.1 Что предоставляет каждый участник

Элемент	Обязательное содержание
Идентификация	ФИО, группа, команда, GitHub-логин и роль
Собственный вклад	2-4 изменения со ссылками на issue, PR, commit и строки diff
Проверка коллеги	ссылка на review, найденная проблема или проверенный критерий
Воспроизведение	команда запуска, SHA-256, границы периодов и результат успешной проверки
Отрицательный сценарий	самостоятельно внесенное нарушение и название сработавшей проверки
Обоснование	правило контракта, риск утечки и ограничение набора

15.2 Демонстрация на защите

Каждый студент без подсказки другого участника должен:

1. получить актуальный tag или commit команды;
2. запустить валидатор на корректном образце;
3. назвать версию контракта и показать SHA-256 в отчете;
4. создать во временной копии одно нарушение и получить ненулевой код завершения;
5. объяснить собственный вклад по diff;
6. объяснить, почему выбранный преподавателем признак доступен или недоступен за 60 минут до отправления.

Преподаватель вправе выбрать другой столбец или нарушение, чем указано в карточке. Демонстрация проводится на собственном окружении студента или в чистом клоне по указанию преподавателя.

15.3 Что не засчитывается как индивидуальный вклад

- только участие в созвоне или распределении задач;
- одинаковый текст evidence card у нескольких студентов;
- ссылка на общий PR без указания собственных изменений;
- commit, полностью созданный другим участником;
- формальное одобрение PR без проверки или комментария;
- демонстрация, которую может выполнить только один член команды;
- исправление данных вручную без автоматического правила и теста.

16 Критерии оценивания

Критерий	Баллы	Признаки полного выполнения
Функциональность	0-3	контракт и валидатор проверяют схему, типы, пропуски, диапазоны, категории и дубликаты; код завершения корректен
Воспроизводимость	0-2	команды документированы, отчет содержит версию и SHA-256, tag `lr2-v1` воспроизводится
Качество и безопасность	0-2	есть карточка, происхождение, ограничения, стратегия против утечки и безопасная политика Git
Командный процесс	0-1	issue, разделение ответственности, PR и содержательное review прослеживаются
Индивидуальная защита	0-2	студент доказывает вклад, выполняет положительную и отрицательную проверку и объясняет решения

Максимум: 10 баллов. Рекомендуемый порог зачета: 6 баллов при отсутствии стоп-факторов.

Стоп-факторы: отсутствие собственного вклада или защиты; публикация закрытых данных или секрета; валидатор всегда возвращает успех; контракт не соответствует фактическим полям; целевая переменная используется как признак; результат невозможно воспроизвести по принятому commit.

17 Контрольный чек-лист команды

- [ ] происхождение, права использования и владелец данных зафиксированы;
- [ ] в Git находится только разрешенный обезличенный образец;
- [ ] карточка данных описывает назначение, состав, ограничения и риски;
- [ ] контракт содержит версию и проверяемые правила для каждого поля;
- [ ] валидатор возвращает код 0 только для корректного набора;
- [ ] JSON-отчет содержит SHA-256 и результаты отдельных проверок;
- [ ] два намеренных нарушения обнаруживаются автоматическими тестами;
- [ ] стратегия разделения и меры против утечки объяснены;
- [ ] CI выполняет lint, тесты и проверку данных до обучения;
- [ ] каждый участник подготовил отдельную evidence card;
- [ ] PR прошел независимое review и создан tag `lr2-v1`.

18 Вопросы для защиты

1. Почему карточка данных не заменяет контракт и наоборот?

2. Почему диапазон нельзя бездумно взять из минимума и максимума текущей выборки?
3. Что изменится в контракте при добавлении необязательного поля?
4. Как SHA-256 помогает воспроизводимости и чего он не гарантирует?
5. Почему `ArrDelay` нельзя использовать в качестве признака?
6. Когда случайное разделение хуже временного или группового?
7. Почему удаление секрета новым commit не устраняет утечку из истории Git?
8. Как валидатор должен вести себя при поврежденном JSON-контракте?
9. Чем предупреждение качества отличается от блокирующего правила?
10. Какие доказательства показывают именно ваш индивидуальный вклад?

19 Рекомендуемая структура результата

```
project/
├── data/
│   ├── contract.json
│   └── flights_sample.csv
├── docs/
│   ├── data-card.md
│   └── evidence/
│   └── lr2-<github-login>.md
├── reports/
│   └── validation/
├── src/mlops_flight_delay/
│   └── validate_data.py
├── tests/
│   └── test_data_contract.py
├── .gitignore
└── README.md
```

Каталог с отчетами может быть исключен из Git, если отчет надежно создается командой и сохраняется как артефакт CI. Решение должно быть единым для команды и описано в README.

20 Источники и дальнейшая работа

- документация Python по `json`, `hashlib` и кодам завершения;
- документация pandas по чтению CSV, пропускам, дубликатам и преобразованию типов;
- документация GitHub по защите чувствительных данных и review pull request;
- Gebru et al. Datasheets for Datasets - подход к документированию происхождения, состава, использования и ограничений набора;
- технологический справочник курса `Технологический стек MLOps 2026-2027.md`.

В ЛР3 контракт и валидатор становятся входным шлюзом DVC-конвейера: команда версионирует происхождение данных и зависимости этапов, не ослабляя принятые в ЛР2 правила.